Classroom Reservation System: Greedy Activity Selection

Cheolwon Park (Student ID: 2024270616)
Korea University
scott9020@korea.ac.kr

1 Web Application and Algorithm Implementation

A classroom reservation system was built using Flask (Python backend) and HTML/CSS/JavaScript with Canvas (frontend). The application lets users add reservation requests or generate random events, then applies the greedy activity selection algorithm to find the maximum set of non-overlapping events.

Steps:

1. **Sort:** Sort all activities by finish time (ascending)
2. **Select:** Pick the first activity; for each subsequent activity, select it if its start ≥ last selected end
3. **Output:** Return the maximum set of non-overlapping reservations

The interface provides an "Add Reservation" form (event name, start, end), a "Generate Random" button, and a "Schedule" button. After scheduling, the app displays a timeline visualization (selected events in blue, rejected in gray), the count of selected vs. total events, and a step-by-step greedy trace table.

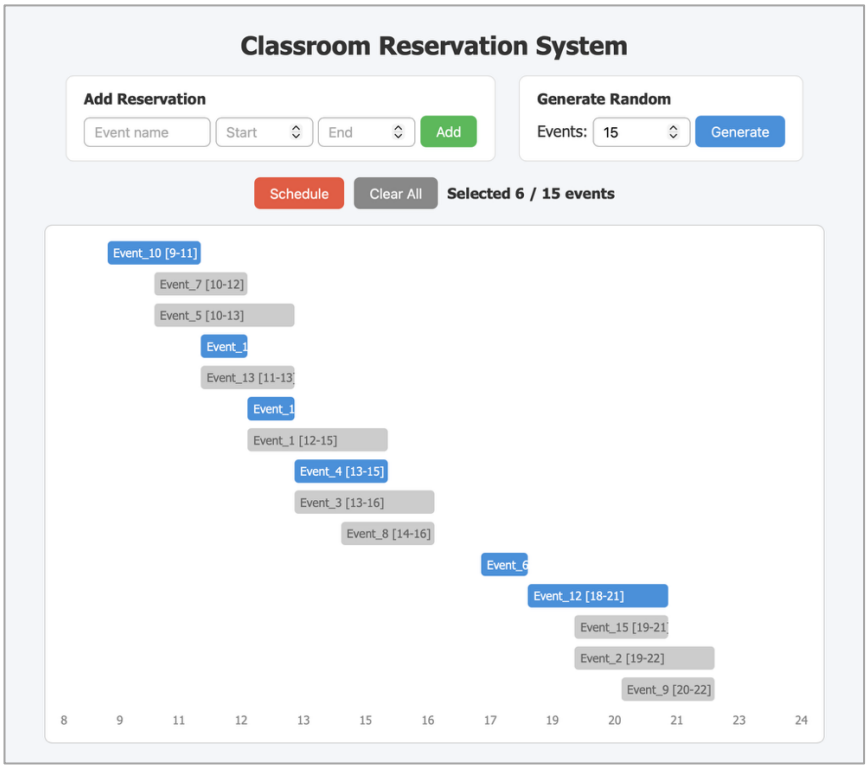

Figure 1. Reservation system with 15 random events after scheduling

2 Step-by-step Greedy Trace

The following trace shows the greedy algorithm applied to 15 randomly generated events (9-22h range), sorted by finish time. `last_end` tracks the end time of the most recently selected event.

Step	Event	Interval	Action	Reason
1	Event_10	[9, 11)	SELECT	start=9 ≥ last_end=−1
2	Event_7	[10, 12)	REJECT	start=10 < last_end=11
3	Event_14	[11, 12)	SELECT	start=11 ≥ last_end=11
4	Event_5	[10, 13)	REJECT	start=10 < last_end=12
5	Event_11	[12, 13)	SELECT	start=12 ≥ last_end=12
6	Event_13	[11, 13)	REJECT	start=11 < last_end=13
7	Event_1	[12, 15)	REJECT	start=12 < last_end=13
8	Event_4	[13, 15)	SELECT	start=13 ≥ last_end=13
9	Event_3	[13, 16)	REJECT	start=13< last_end=15
10	Event_8	[14, 16)	REJECT	start=14 < last_end=15
11	Event_6	[17, 18)	SELECT	start=17 ≥ last_end=15
12	Event_12	[18, 21)	SELECT	start=18 ≥ last_end=18
13	Event_15	[19, 21)	REJECT	start=19 < last_end=21
14	Event_2	[19, 22)	REJECT	start=19 < last_end=21
15	Event_9	[20, 22)	REJECT	start=20 < last_end=21

Table 1. Greedy trace on 15 sample events – 6 selected out of 15

3 Analysis

3.1 Why does sorting by finish time guarantee the optimal solution?

Sorting by finish time ensures that at each step we select the activity that ends earliest, leaving the maximum remaining time for subsequent activities. This is optimal because of the **greedy-choice property**: there always exists an optimal solution that includes the activity with the earliest finish time. Proof sketch: if an optimal solution O does not include the earliest-finishing activity a, we can replace the first activity in O with a (since a ends no later), and the result is still valid and has the same size. Repeating this exchange argument for each greedy choice shows the algorithm produces an optimal solution.

3.2 Could sorting by start time or duration work instead?

Sorting by start time does not work. Counter-example: activities [0, 100), [1, 3), [4, 6). Sorting by start time selects [0, 100) first, yielding only 1 event, while the optimal is 2: {[1, 3), [4, 6)}. The earliest-starting activity can be arbitrarily long, blocking many shorter events.

Sorting by duration also fails. Counter-example: activities [0, 4), [3, 6), [5, 9). Sorting by duration selects [3, 6) first (duration 3), which blocks both [0, 4) and [5, 9). The optimal is 2: {[0, 4), [5, 9)}. Short activities in the middle can overlap with multiple non-overlapping events on either side.